A* algorithm

Tran, Van Hoai

E-mail: hoai@hcmut.edu.vn

2024-2025 - Semester 2

Outline

1 Shortest path problem

2 A* algorithm

3 Road networks

Outline

1 Shortest path problem

2 A* algorithm

3 Road networks

The shortest path problem

Given a weighted graph $G(V, E, w)$, in which:

- V - set of vertices
- E - set of edges (directed or undirected)
- $w : E \to \mathbb{R}_+$ - weights on edges

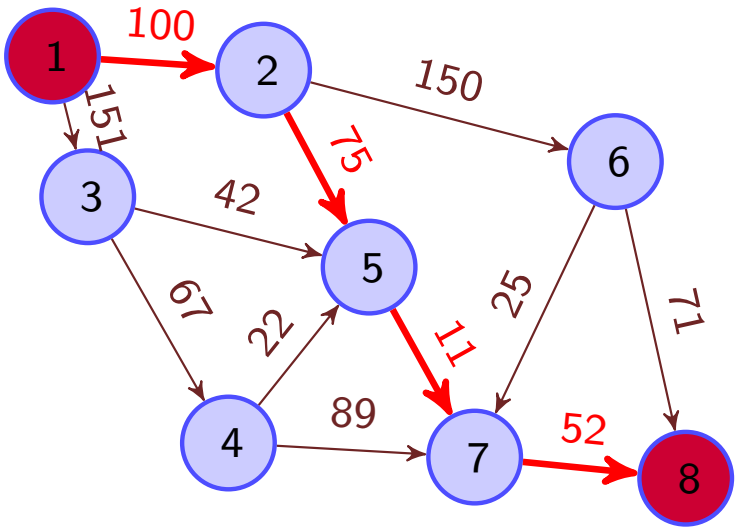
The shortest path problem

Finding a path between two vertices in G such that the sum of the weights of its constituent edges is minimized.

Related fundamental studies

- Graph traversal
- Path-finding
- Mathematical programming: modeling, numerical optimization
- Randomized algorithms
- ...

The shortest path problem: an example



Minimize $100x_{12} + 151x_{13} + \ldots + 52x_{78}$

Subject to

$$
\begin{aligned}
x_{12} + x_{13} &= 1 \\
-x_{12} + x_{25} + x_{26} &= 0 \\
-x_{13} + x_{34} + x_{35} &= 0 \\
-x_{34} + x_{45} + x_{47} &= 0 \\
-x_{25} - x_{35} - x_{45} + x_{57} &= 0 \\
-x_{26} + x_{67} + x_{68} &= 0 \\
-x_{47} - x_{57} - x_{67} + x_{78} &= 0 \\
-x_{68} - x_{78} &= -1
\end{aligned}
$$

$x_{ij} \in \{0, 1\}$

State-of-the-art algorithms

- Single-source

  Dijkstra $O(V^2)$ (1959), Bellman-Ford $O(VE)$ (1958), Johnson $((E + V) \log V)$ (1977), ..., Duan et. al. $O(E\sqrt{\log V \log\log V})$ (**2023**).

- Multiple-sources

  Floyd-Warshall $O(V^3)$ (1962)

Outline

1 Shortest path problem

2 A* algorithm

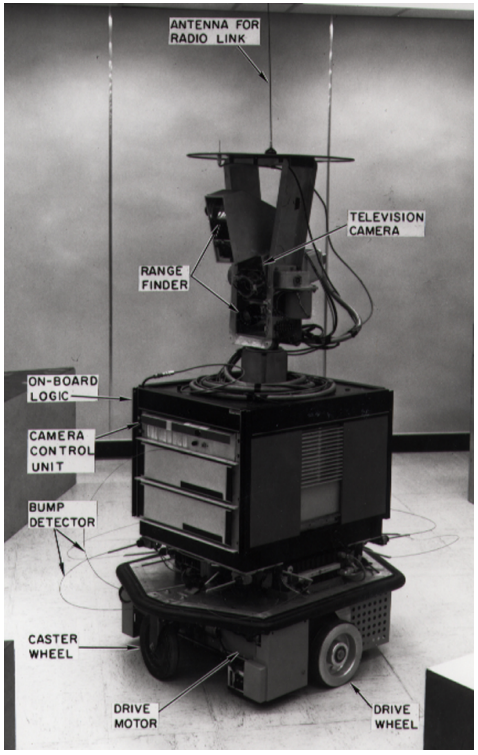
3 Road networks

A* motivation

Why A*?

- To avoid search all possibilities to find the optimal.
- To require a guide for the search

  $f(v) = g(v) + h(v)$, in which
  - $f(v)$: path weight from the source to destination (thru v).
  - $g(v)$: path weight from the source to v.
  - $h(v)$: **estimate** of path weight from v to destination.
- $h(n)$: availability in practical applications



Robot of Shakey project (wiki)

A* algorithm

- Hart et.al. published in 1968, $O(E \log V)$.
- No shortest path tree generated.

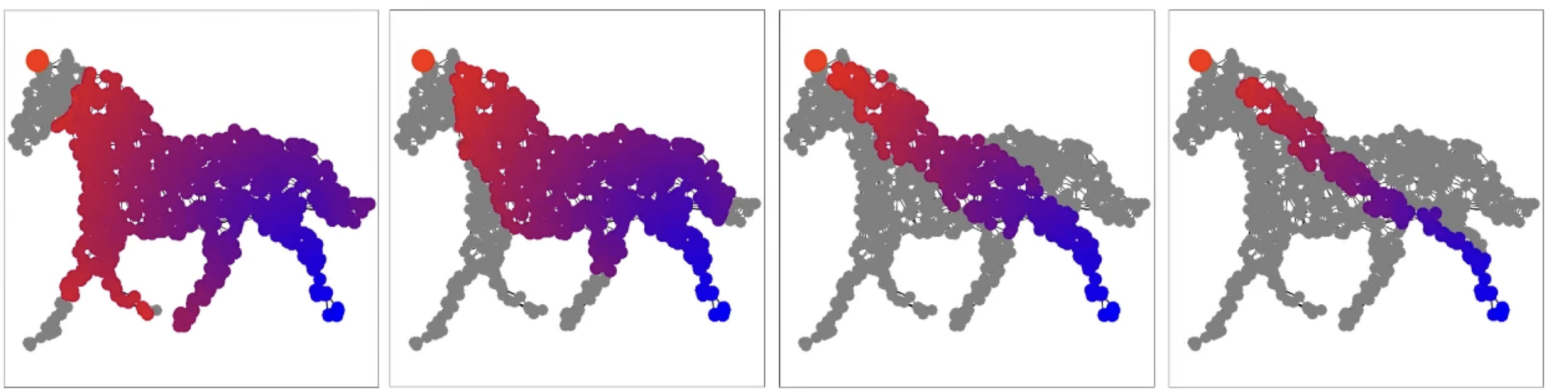
Dijkstra algorithm

```
procedure Dijkstra( G, s, d )
  // Initialization step
  forall vertices v
    Label[v] := ∞
    Prev[v] := −1
  endfor
  Label[s] := 0 // s is the source
  S := ∅

  // Iteration step
  while d ∉ S // d is the destination
     u := arg min_{v∈V\S}{Label[v]}
     S := S ∪ {u}
     forall neighbor vertices v of u, and not in S
       if (Label[u] + w(u,v)) < Label(v)
       begin
         Label[v] := Label[u] + w(u,v)
         Pred[v] := u
       end
  endwhile
```

Comments

- Label[v] = $g(v)$
- Do not know which vertex is next, just depending on smallest $g(.)$.



A* algorithm

```
procedure Astar( G, s, d )
  // Initialization step
  forall vertices v
    Label[v] := ∞
    Est[v] := ∞
    Prev[v] := −1
  endfor
  Label[s] := 0
  Est[s] := h(s)
  C := ∅
  O := {s}

  // Iteration step
  while O ≠ ∅
     u := arg min_{v∈O}{Label[v] + Est[v]} // lowest f(.)
     if (u == d)
     begin
       reconstruct the path
       return
     end
     C := C ∪ {u}
     forall neighbor vertices v of u and not in C
       if (Label[u] + w(u, v)) < Label(v)
       begin
         Label[v] := Label[u] + w(u, v)
         Est[v] := h(v)
         Pred[v] := u
         O := O ∪ {v}
       end
  endwhile
```

A* algorithm

```
procedure Astar( G, s, d )
  // Initialization step
  forall vertices v
    Label[v] := ∞
    Est[v] := ∞
    Prev[v] := −1
  endfor
  Label[s] := 0
  Est[s] := h(s)
  C := ∅
  O := {s}

  // Iteration step
  while O ≠ ∅
     u := arg min_{v∈O}{Label[v] + Est[v]} // lowest f(.)
     if (u == d)
     begin
       reconstruct the path
       return
     end
     C := C ∪ {u}
     forall neighbor vertices v of u and not in C
       if (Label[u] + w(u, v)) < Label(v)
       begin
         Label[v] := Label[u] + w(u, v)
         Est[v] := h(v)
         Pred[v] := u
         O := O ∪ {v}
       end
  endwhile
```

Comments

- Please check the correctness of the left algorithm :-((I've just prepared it in this Morning)
- What if $h(.)$ is dynamic, uncertain?
- What if $f(.)$ is a vector function?

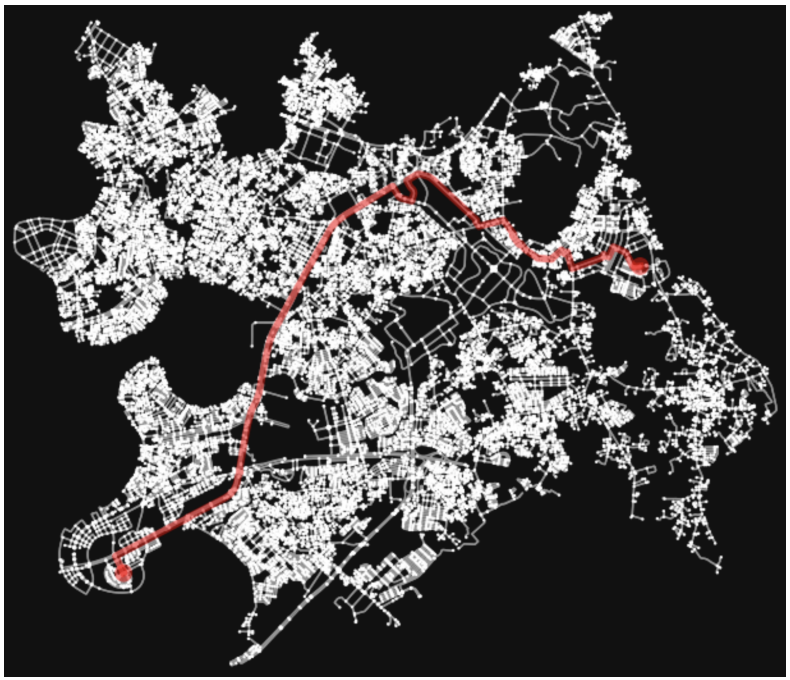
Outline

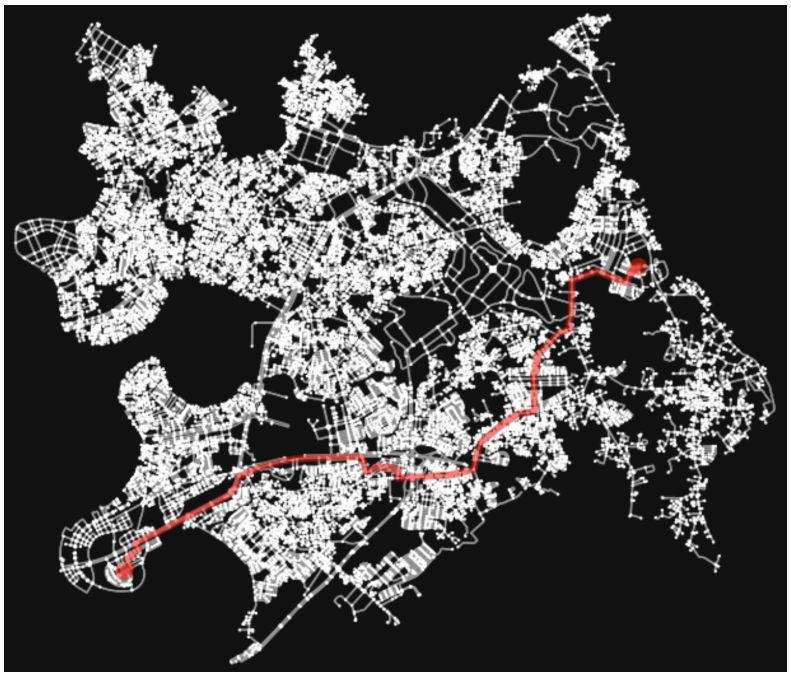
1 Shortest path problem

2 A* algorithm

3 Road networks

Road network and A* (not yet verified code)





Topics for assignment 2

- **Priority queue** with different synchronization schemes
- Contraction hierarchies
- Pruning
- Labeling
- ...